

DATABASE APPLICATION DEVELOPMENT



Objectives

- ▶ Define terms
- ▶ Explain three components of client/server systems: presentation, processing, and storage
- ▶ Distinguish between two-tier and three-tier architectures



Objectives

- ▶ Describe how to connect to databases in 2-tier systems using VB.NET and Java
- ▶ Describe key components and information flow in Web applications
- ▶ Describe how to connect to databases in 3-tier applications using JSP, PHP, and ASP.NET



Objectives

- ▶ Explain the purpose of XML
- ▶ See how XQuery can be used to query XML documents
- ▶ Explain how XML fosters Web services and SOAs



Client/Server Architectures

- ▶ Networked computing model
- ▶ Processes distributed between clients and servers
- ▶ Client–Workstation (usually a PC) that requests and uses a service
- ▶ Server–Computer (PC/mini/mainframe) that provides a service
- ▶ For DBMS, server is a database server



Application Logic in Client/Server Systems

Presentation Logic

Input-keyboard/mouse
Output-monitor/printer

Processing Logic

I/O processing
Business rules
Data management

Storage Logic

Data storage/retrieval

GUI Interface

**Procedures, functions,
programs**

DBMS activities



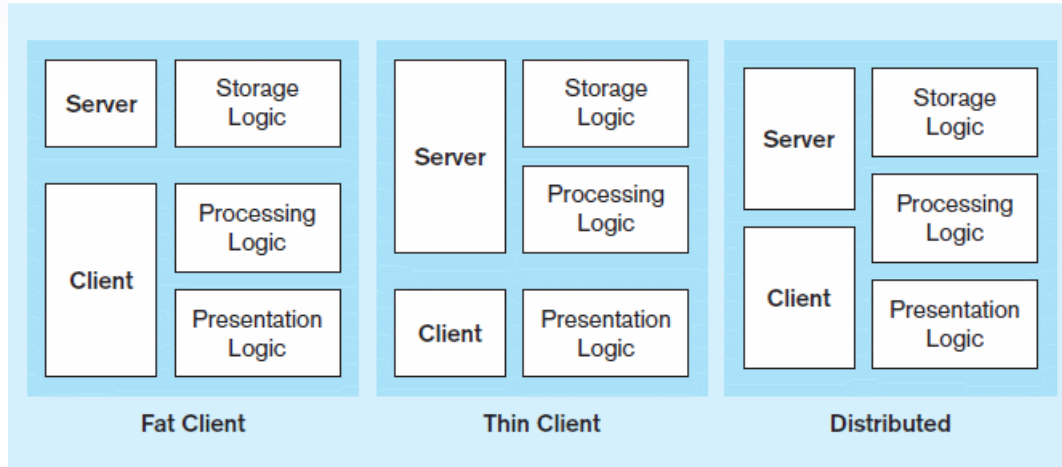
Application Partitioning

- ▶ Placing portions of the application code in different locations (client vs. server) after it is written
- ▶ Advantages
 - ▶ Improved performance
 - ▶ Improved interoperability
 - ▶ Balanced workloads



Common Logic Distribution

a) Two-tier client-server environments

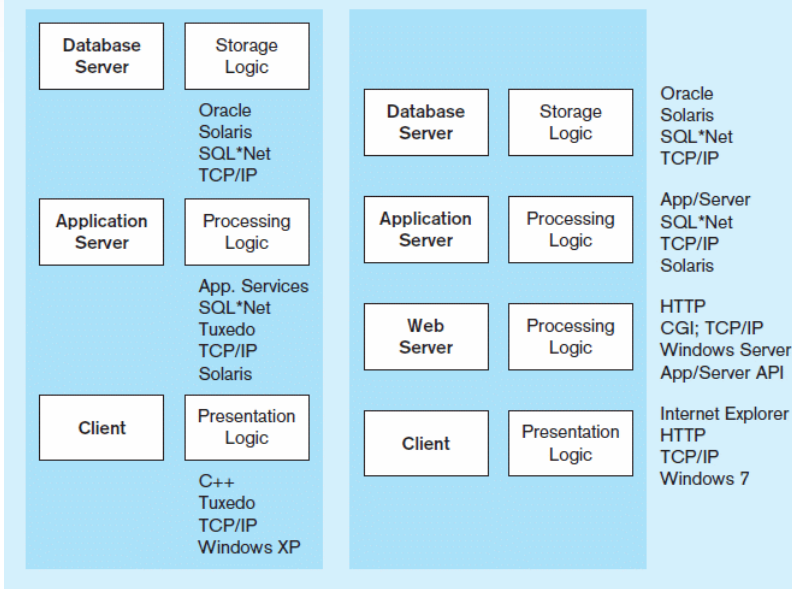


Processing logic could be at client (fat client), server (thin client), or both (distributed environment).



Common Logic Distribution

b) Three-tier and n -tier client-server environments



Processing logic will be at application server or Web server.



Two-Tier Database Server Architectures

- ▶ Client workstation is responsible for
 - ▶ Presentation logic
 - ▶ Data processing logic
 - ▶ Business rules logic



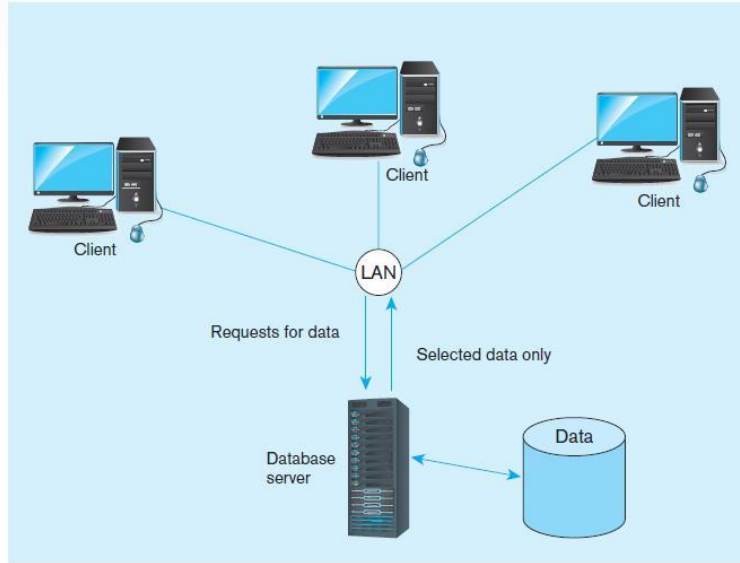
Two-Tier Database Server Architectures

- ▶ Server performs all data storage, access, and processing
 - ▶ Typically called a database server
 - ▶ DBMS is only on server



Two-Tier Database Server Architectures

Front-end programs



Back-end functions



Characteristics of Two-Tier Client/Server Systems

- ▶ Departmental in scope (few users)
- ▶ Not mission-critical
- ▶ Low transaction volumes
- ▶ Common programming languages:
 - ▶ Java, VB .NET, C#
- ▶ Interface database via middleware, APIs



► Middleware and APIs

- ▶ **Middleware** – software that allows an application to interoperate with other software without requiring user to understand and code low-level operations
- ▶ **Application Program Interface (API)** – routines that an application uses to direct the performance of procedures by the computer's operating system
- ▶ **Common database APIs** – ODBC, ADO .NET, JDBC



Steps for Using Databases via Middleware APIs

- ▶ Identify and register a database driver.
- ▶ Open a connection to a database.
- ▶ Execute a query against the database.
- ▶ Process the results of the query.
- ▶ Repeat steps 3–4 as necessary.
- ▶ Close the connection to the database.



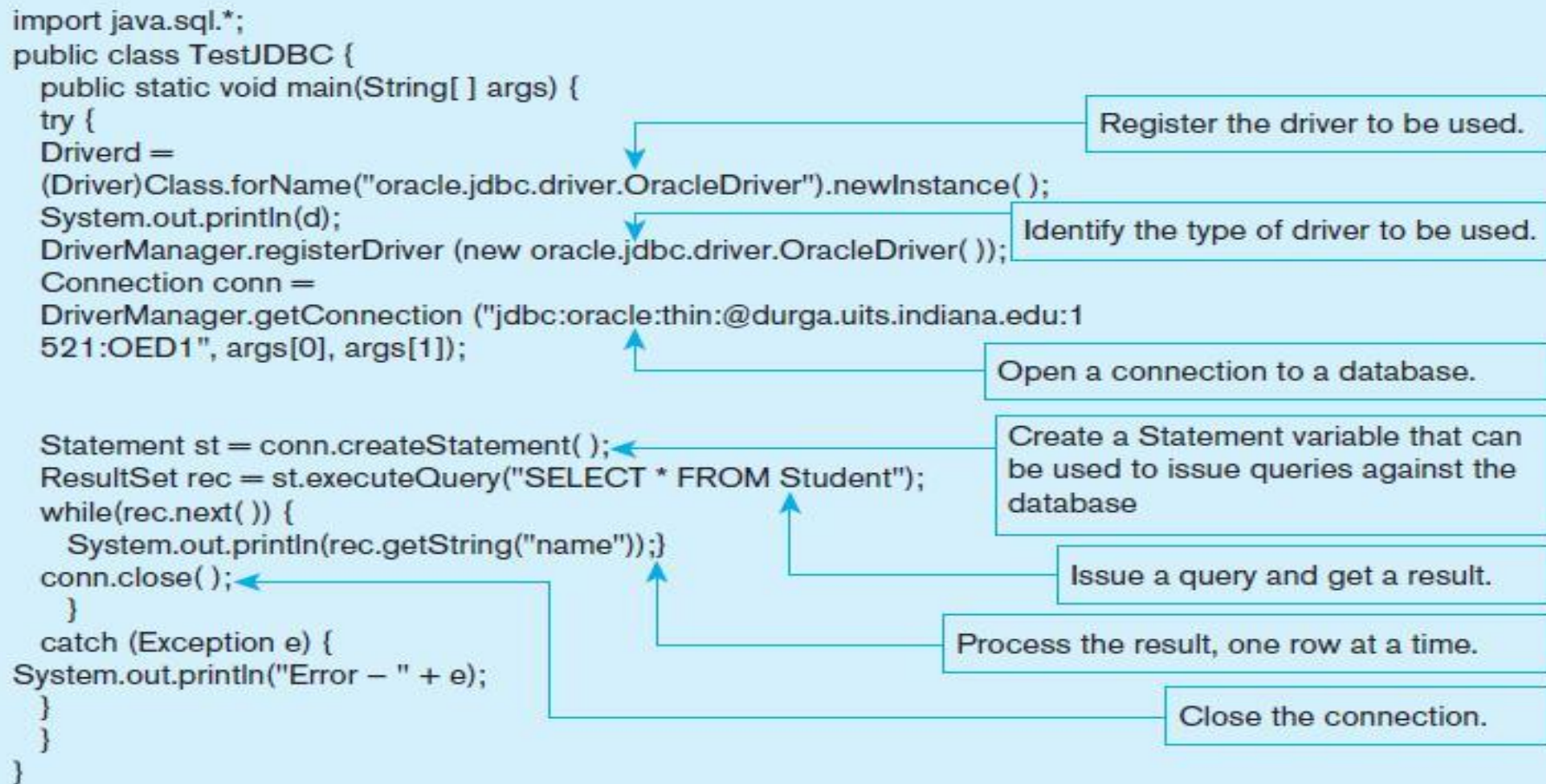


FIGURE 8-5 Database access from a Java program

Three-Tier Architectures

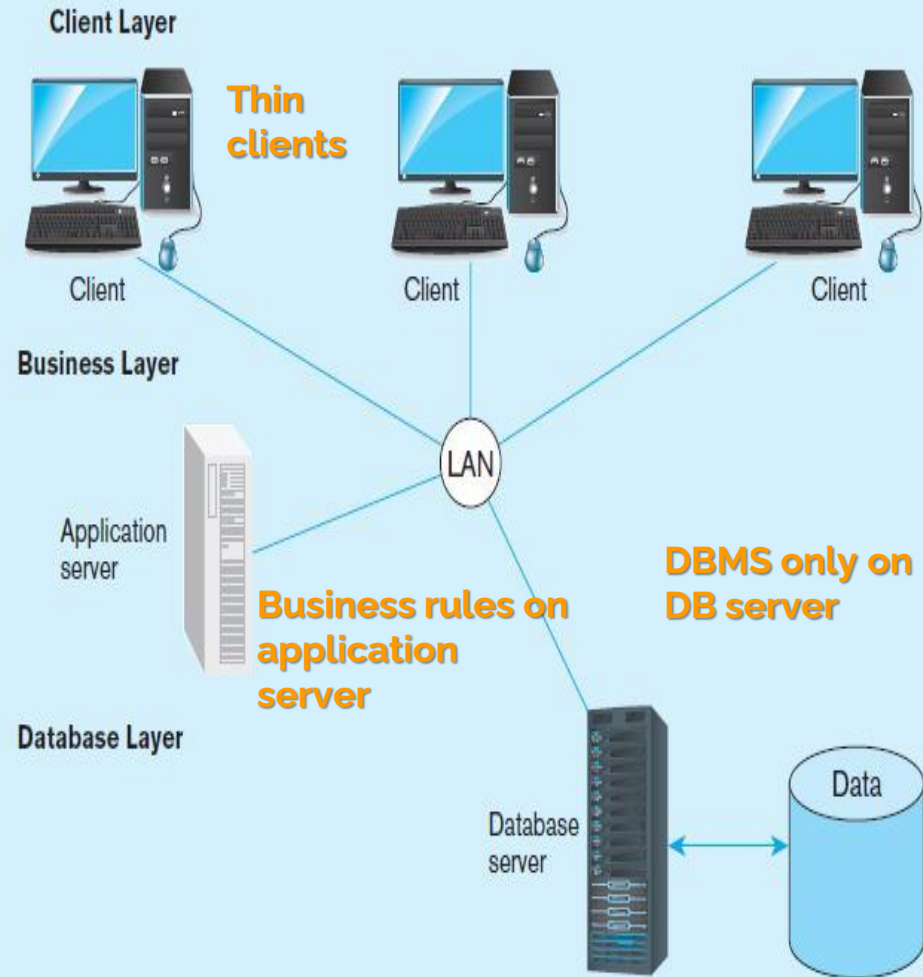
Client	GUI interface (I/O processing)	<i>Browser</i>
Application server	Business rules	<i>Web Server</i>
Database server	Data storage	<i>DBMS</i>

Thin Client

- PC just for user interface and a little application processing. Limited or no data storage (sometimes no hard drive)



Generic Three-Tier Architecture

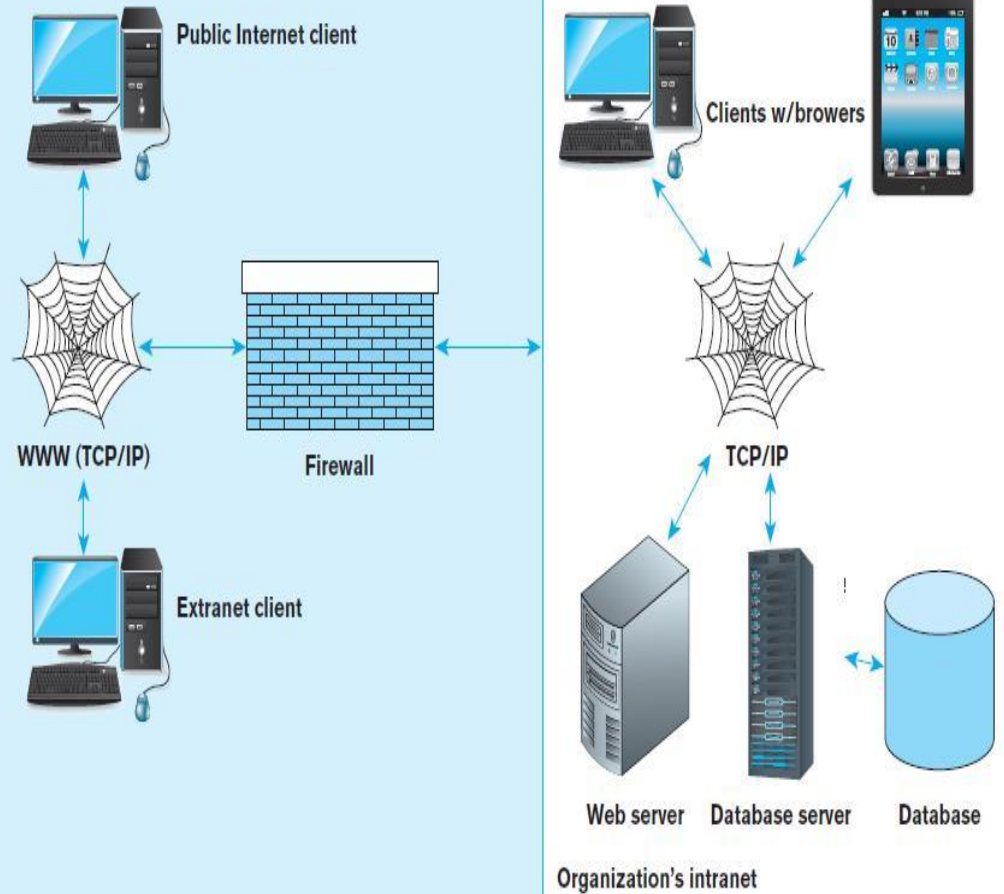


Thin Client

- ▶ An application where the client (PC) accessing the application primarily provides the user interfaces and some application processing, usually with no or limited local data storage.
- ▶ Usually, thin client application is a Web browser and the 3-tier architecture involves a Web application.



Database-enabled Internet/Intranet Environment



Web Application Components

- ▶ Database server – hosts the DBMS
 - ▶ e.g. Oracle, SQL Server, Informix, MS Access, MySql
- ▶ Web server – receives and responds to browser requests using HTTP protocol
 - ▶ e.g. Apache, Internet Information Services (IIS)



Web Application Components

- ▶ **Application server – software building blocks for creating dynamic web sites**
 - ▶ e.g. MS ASP .NET framework, Java EE, ColdFusion, PHP
- ▶ **Web browser – client program that sends web requests and receives web pages**
 - ▶ e.g. Internet Explorer, Firefox, Safari, Google Chrome



Languages for Creating Web Pages

- ▶ **Hypertext Markup Language (HTML)**
 - ▶ Markup language specifically for Web pages
- ▶ **Standard Generalized Markup Language (SGML)**
 - ▶ Markup language standard
- ▶ **Extensible Markup Language (XML)**
 - ▶ Markup language allowing customized tags



Languages for Creating Web Pages

- ▶ XHTML
 - ▶ XML-compliant extension of HTML
- ▶ JavaScript/VBScript
- ▶ Scripting languages that enable interactivity in HTML documents



Languages for Creating Web Pages

- ▶ **Cascading Style Sheets (CSS)**
 - ▶ Control appearance of Web elements in an HTML document
- ▶ **XSL and XSLT**
 - ▶ XMS style sheet and transformation to HTML



Processing in 3-Tier Applications

- ▶ **Static page requests**
 - ▶ **.htm or .html requests handled by the Web server**



Processing in 3-Tier Applications

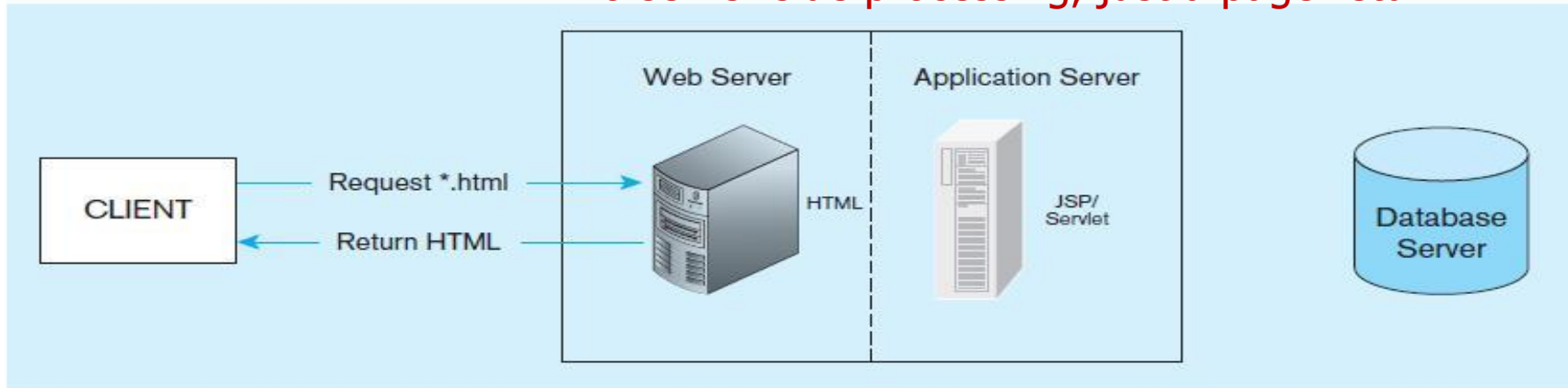
- ▶ **Dynamic page requests**
 - ▶ .jsp, .aspx, and .php requests are routed to the application server
 - ▶ Server-side processing by JSP servlet, ASP .NET application, ColdFusion, or PHP
 - ▶ Database access via JDBC, ADO .NET, or other database middleware



Information flow in a three-tier architecture

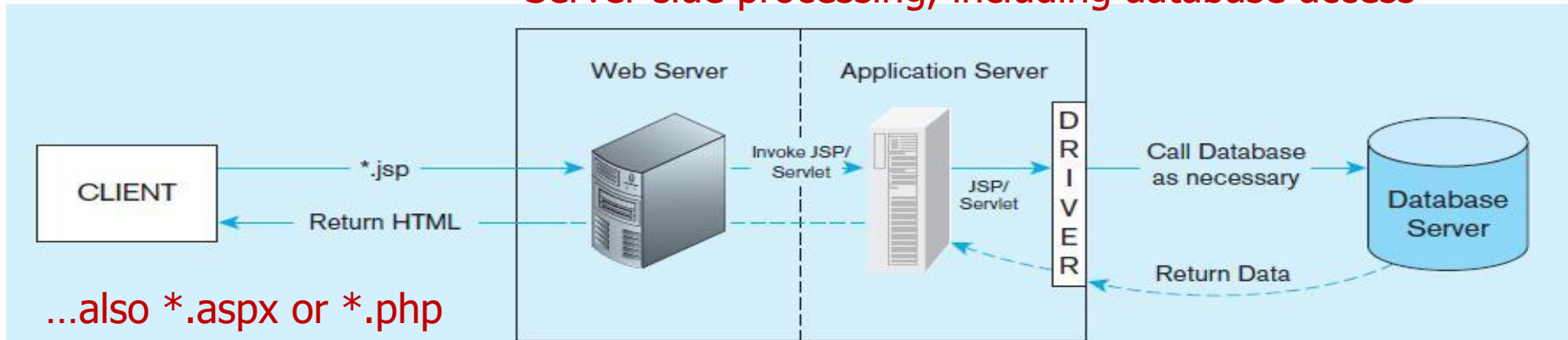
(a) Static page request

No server side processing, just a page return



(b) Dynamic page request

Server side processing, including database access



Considerations in 3-Tier Applications

- ▶ **Stored procedures**
 - ▶ Code logic embedded in DBMS
 - ▶ Improve performance, but proprietary
- ▶ **Transactions**
 - ▶ Involve many database updates
 - ▶ Either all must succeed, or none should occur



Considerations in 3-Tier Applications

- ▶ Database connections
 - ▶ Maintaining an open connection is resource-intensive
 - ▶ Use of connection pooling



Benefits of Stored Procedures

- ▶ Performance improves for compiled SQL statements
- ▶ Reduced network traffic
- ▶ Improved security
- ▶ Improved data integrity
- ▶ Thinner clients



Benefits of Three-Tier Architectures

- ▶ Scalability
- ▶ Technological flexibility
- ▶ Long-term cost reduction
- ▶ Better match of systems to business needs
- ▶ Improved customer service
- ▶ Competitive advantage
- ▶ Reduced risk



Cloud computing

- ▶ A model for creating ubiquitous, convenient, on-demand access to network services
- ▶ Characteristics: on-demand, broad network access, resource pooling, rapid elasticity, measured service



Cloud computing

- ▶ Types of cloud computing:
 - ▶ Infrastructure-as-a-service (IaaS)
 - ▶ Platform-as-a-service (PaaS)
 - ▶ Software-as-a-service (SaaS)



Extensible Markup Language (XML)

- ▶ A text-based markup language (like HTML)
 - ▶ Uses elements, tags, attributes
 - ▶ Includes document type declarations (DTDs), XML schemas, comments, and entity references
- ▶ Revolutionizes the way data are exchanged over the Internet



Extensible Markup Language (XML)

- ▶ Document Structure Declarations (DSD), XML Schema (XSD) and Relax NG replacing DTDs for validating XML document structure
- ▶ XSD – language for defining XML databases, recommended by the W3C



Sample XML Schema (XSD)

Schema is a record definition, analogous to the Create SQL statement, and therefore provides metadata.

```
<?xml version="1.0" encoding="utf-8" ?>
<xsd:schema id="salespersonSchema"
xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:element name="Salesperson" type="SalespersonType" />
  <xsd:complexType name="SalespersonType">
    <xsd:sequence>
      <xsd:elementname="SalespersonID"
        type="xsd:integer"/>
      <xsd:elementname="SalespersonName"
        type="xsd:string" />
      <xsd:element name="SalespersonTelephone"
        type="PhoneNumberType">
      <xsd:element name="SalespersonFax"
        type="PhoneNumber" minOccurs="0" />
    </xsd:element>
  </xsd:sequence>
</xsd:complexType>
<xsd:simpleType name="PhoneNumberType">
  <xsd:restriction base="xsd:string">
    <xsd:length value="12" />
    <xsd:pattern value="\d{3}-\d{3}-\d{4}" />
  </xsd:restriction>
</xsd:simpleType>
</xsd:schema>
```

Sample XML Document Data

This XML data conforms to the XML schema of the previous slide, and involves elements and attributes defined in the schema.

This is analogous to a record in a database.

```
<?xml version="1.0" encoding="utf-8" ?>
<Salesperson xmlns:xsi=http://www.w3.org/2001/XMLSchema-instance
xsi:noNamespaceSchemaLocation="salespersonSchema.xsd">
  <SalespersonID>1</SalespersonID>
  <SalespersonName>Doug Henny</SalespersonName>
  <SalespersonTelephone>813-444-5555</SalespersonTelephone>
</Salesperson>
```

Another Sample XML Document

```
<?xml version = "1.0"/>
<furniturecompany>
  <product ID="1">
    <description>End Table</description>
    <finish>Cherry</finish>
    <standard price>175.00</standard price>
    <line>1</line>
  </product>
</furniturecompany>
```

Storing XML Documents

- ▶ Storing as files introduces the same file processing problems stated in Ch 1
- ▶ Four common options:
 - ▶ Store XML data in a relational database by shredding the XML document
 - ▶ Store entire XML document in a large field (BLOB or CLOB)
 - ▶ Store the XML document using special XML columns
 - ▶ Store the XML document using a native XML database (not relational)



Retrieving XML Documents

- ▶ XPath – One of a set of XML technologies supporting XQuery development, locating data in XML documents
- ▶ XQuery – An XML transformation language that allows applications to query both relational databases and XML data
- ▶ Sample XQuery expression:

```
for $p in doc("PVFC.xml")/furniture company/product
where $p/standardprice>300.00
order by $p/description
return $p/description
```

▶ Displaying XML Data

- ▶ **Extensible Stylesheet Language Transformation (XSLT)** – A language used to transform complex XML documents and also used to create HTML pages from XML documents
- ▶ XSLT can translate a single XML document into both standard HTML and WAP/WML for cell phones without the necessity for two different pages

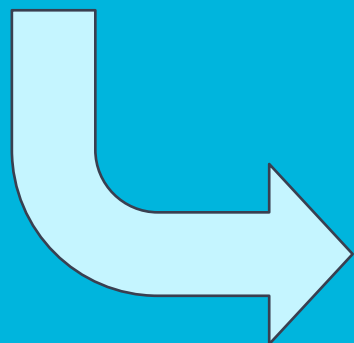


XSLT Code

```
<?xml version = "1.0"?>
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
<xsl:template match="/">
  <html>
    <body>
      <h2>Product Listing</h2>
      <table border="1">
        <tr bgcolor="orange">
          <th>Description</th>
          <th>Finish</th>
          <th>Price</th>
        </tr>
        <xsl:for-each select="furniturecompany/product">
          <tr>
            <td><xsl:value-of select="description"/></td>
            <td><xsl:value-of select="finish"/></td>
            <td><xsl:value-of select="price"/></td>
          </tr>
        </xsl:for-each>
      </table>
    </body>
  </html>
</xsl:template>
</xsl:stylesheet>
```

```
<furniture company>
  <product ID="1">
    <description>End Table</description>
    <finish>Cherry</finish>
    <standard price>175.00</standard price>
    <line>1</line>
  </product>
  <product ID="2">
    <description>Coffee Table</description>
    <finish>Natural Ash</finish>
    <standard price>200.00</standard price>
    <line>2</line>
  </product>
</furniture company>
```

When applied to the above XML data, the XSLT code from Figure 8-15b produces the display on the right.



Product Listing

Description	Finish	Price
End Table	Cherry	175.00
Coffee Table	Natural Ash	200.00

XML and Web Services

- ▶ **Web Services** – a set of emerging XML-based standards that define protocols for automatic communication between software programs over the Web
- ▶ **Universal Description, Discovery, and Integration (UDDI)** – standard for creating and distributing Web services



XML and Web Services

- ▶ **Web Services Description Language (WSDL)** – XML-based grammar for describing a Web Service and specifying its public interface
- ▶ **Simple Object Access Protocol (SOAP)** – XML-based communication protocol for sending messages between applications over the Internet



Web Services Protocol Stack

Publish, Find, Use Services	UDDI	Universal Description, Discovery, Integration
Describe Services	WSDL	Web Services Description Language
Service Interactions	SOAP	Simple Object Access Protocol
Data Format	XML	eXtensible Markup Language
Open Communications	Internet	

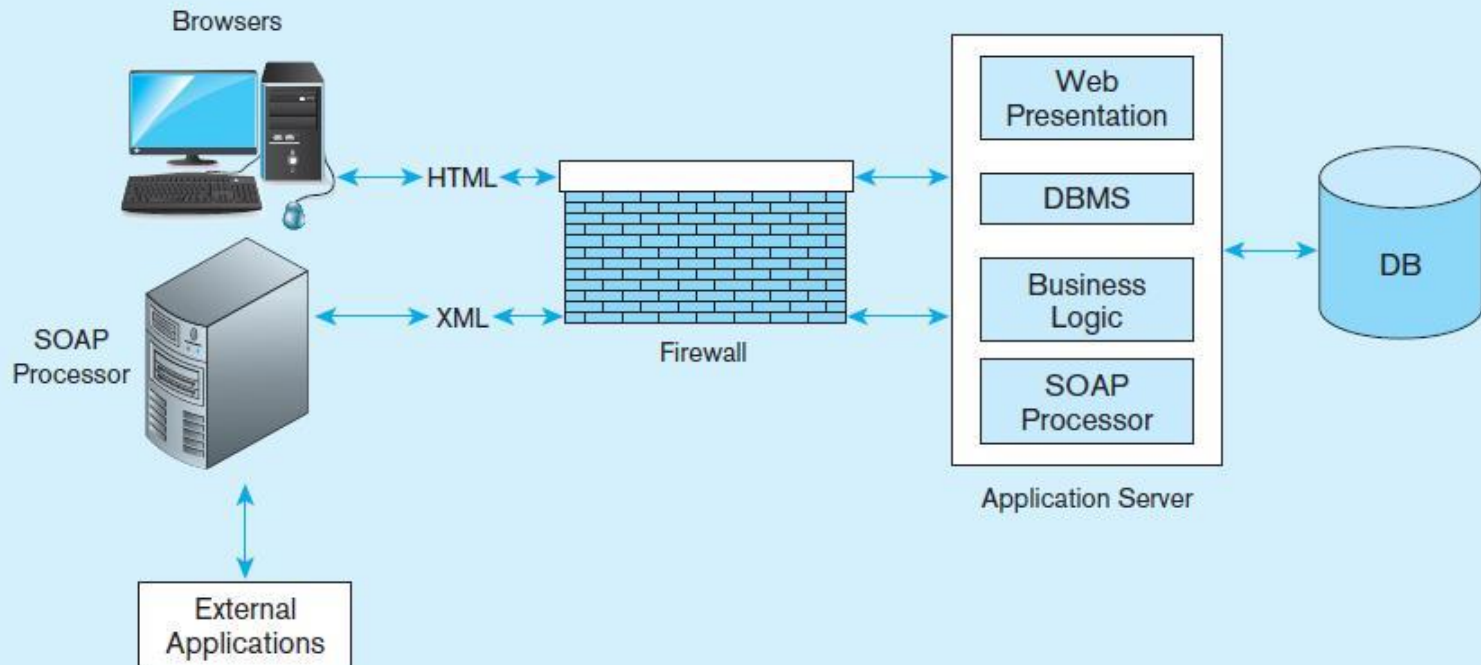
SOAP request sent from customer to supplier

```
<soap:Envelope xmlns:soap=http://schemas.xmlsoap.org/soap/envelope/>
  <soap:Body>
    <getProductDetails xmlns=http://supplier.example.com/ws
      <productID>32879</productID>
    </getProductDetails>
  </soap:Body>
</soap:Envelope>
```

SOAP response sent from supplier to customer

```
<soap:Envelope xmlns:soap=http://schemas.xmlsoap.org/soap/envelope/>
  <soap:Body>
    <getProductDetailsResponse xmlns="suppliers.example.com/ws">
      <getProductDetailsResult>
        <productName>Dining Table</productName>
        <Finish>Natural Ash</Finish>
        <Price>800</Price>
        <inStock>True</inStock>
      </getProductDetailsResult>
    </getProductDetailsResponse>
  </soap:Body>
</soap:Envelope>
```


Web services deployment



Service Oriented Architecture (SOA)

- ▶ A collection of services that communicate with each other, usually by passing data or coordinating a business activity
- ▶ A new paradigm for IT application development, based mostly on Web services
- ▶ Loosely coupled, highly interoperable components
- ▶ Leads to flexibility and shorter development time

